

ARM Cryptographic Extension을 활용한 Rijndael-256 구현

<https://youtu.be/mbFTdKw8hRg>

- Software Optimization of Rijndael256 for Modern x86-64 Platforms
 - 대표적인 AES 블록 암호를 위해서 하드웨어 가속기 AES-NI가 존재.
 - 이를 통해서 AES는 굉장히 빠르게 암호화를 할 수 있음
- 하지만 Rijndael-256은 AES 알고리즘과 다른 부분이 있음
 - AES는 Shiftrows(0, 1, 2, 3) 하도록 되어있음
 - Rijndael-256은 Shiftrows(0,1,3,4) 으로 바뀜
 - 따라서,
 - AES-NI는 128비트 레지스터를 그대로 사용하도록 되어있음
 - 예를 들면 -> Aesenc xmm1, xmm1
 - 근데, 블록크기가 256비트라면 state가 두개의 레지스터로 나뉘어 저장해야함
 - Aesenc xmm1, xmm1 / aesenc xmm2, xmm2
 - 근데 Shiftrows가 달라서 aesenc를 사용할 수 없음

• AES의 라운드 함수

- Subbyte : 바이트 독립적
- Shiftrows : 행 단위의 순열
- Mixcolumns : 열 단위의 확산
- Addroundkey : 바이트 독립적

Mixcolumns

>>> 1

>>> 2

>>> 3

Shiftrows

A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B

xmm1				xmm2			
A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B

>>> 1
>>> 3
>>> 4

xmm1				xmm2			
A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B
A	A	A	A	B	B	B	B

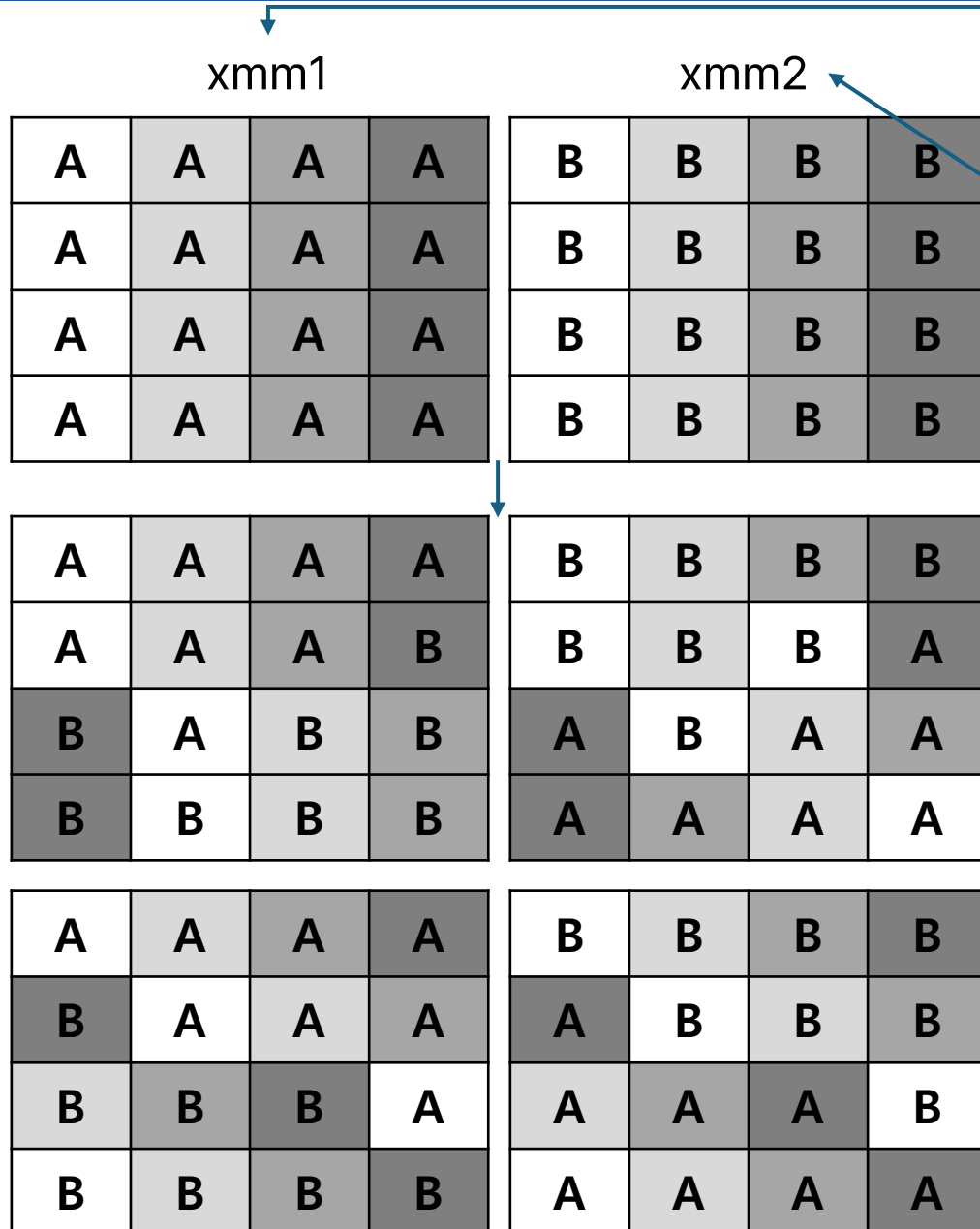
Shiftrows

xmm1				xmm2			
A	A	A	A	B	B	B	B
B	A	A	A	A	B	B	B
B	B	B	A	A	A	A	B
B	B	B	B	A	A	A	A

<<< 1
<<< 2
<<< 3

xmm1				xmm2			
A	A	A	A	B	B	B	B
A	A	A	B	B	B	B	A
B	A	B	B	A	B	A	A
B	B	B	B	A	A	A	A

```
vpblendvb %xmm2, %xmm1, %xmm5, %xmm3
vpblendvb %xmm1, %xmm2, %xmm5, %xmm4
pshufb %xmm8, %xmm3
pshufb %xmm8, %xmm4
aesenc %xmm6, %xmm3
aesenc %xmm7, %xmm4
```



```
.Lround_loop:
/* Pre-shuffle for Rijndael-256 ShiftRows */

/* Prepare L_input */
movdqa xmm2, xmm0
pshufb xmm2, xmm8      /* L's contribution */
movdqa xmm3, xmm1
pshufb xmm3, xmm9      /* R's contribution */
por xmm2, xmm3          /* L_input = L_part | R_part */

/* Prepare R_input */
movdqa xmm3, xmm1
pshufb xmm3, xmm10     /* R's contribution */
movdqa xmm4, xmm0
pshufb xmm4, xmm11     /* L's contribution */
por xmm3, xmm4          /* R_input = R_part | L_part */

/* Apply aesenc (SubBytes + ShiftRows + MixColumns + AddRoundKey) */
aesenc xmm2, [rbx]      /* aesenc(L_input, roundKey_L) */
aesenc xmm3, [rbx + 16] /* aesenc(R_input, roundKey_R) */

/* Update state */
movdqa xmm0, xmm2      /* state_L = result_L */
movdqa xmm1, xmm3      /* state_R = result_R */

/* Next round */
add rbx, 32
sub eax, 1
jnz .Lround_loop
```

ARM
Cryptographic Extension

AESE v0, v0
AESMC v0, v0
AESE v1, v1
AESMC v1, v1

• 벤치마크 결과

구현	시간 (ms)	처리량 (MB/s)	Naive 대비 속도
Naive	527.03	60.72	1.0x
T-table	209.35	152.85	2.5x
AES-NI	20.55	1,557.49	25.7x

Lenc_round_loop:

```
/* Pre-shuffle for Rijndael-256 ShiftRows (0,1,3,4) */
/* TBL uses {v0, v1} as 32-byte table */
tbl v2.16b, {v0.16b, v1.16b}, v16.16b // L_shuffled
tbl v3.16b, {v0.16b, v1.16b}, v17.16b // R_shuffled
```

```
/* Load round keys for this round */
ld1 {v20.16b, v21.16b}, [x3], #32
```

```
/*
 * AESE performs: SubBytes(ShiftRows(state XOR key))
 * With zero key: SubBytes(ShiftRows(state))
 * Then AESMC performs: MixColumns
 * Finally XOR round key (must be AFTER MixColumns!)
 */
aese v2.16b, v31.16b // SubBytes(ShiftRows(L_shuffled))
aese v3.16b, v31.16b // SubBytes(ShiftRows(R_shuffled))
```

```
/* MixColumns FIRST */
aesmc v0.16b, v2.16b // state_L = MixColumns(result_L)
aesmc v1.16b, v3.16b // state_R = MixColumns(result_R)
```

```
/* XOR round key AFTER MixColumns (matching x86 aesenc behavior) */
eor v0.16b, v0.16b, v20.16b
eor v1.16b, v1.16b, v21.16b
```

```
/* Loop control */
subs w4, w4, #1
b.ne Lenc_round_loop
```

벤치마크 요약 (32MB 데이터, 256-bit 키)

구현	시간	처리량	속도
Native C	314ms	101 MB/s	1.0x
T-table	158ms	202 MB/s	2.0x
ARM Crypto	15ms	2,100 MB/s	21x

AES-NI	1563 MB/s (1위)
Bitslice-8	230 MB/s (2위)
T-table	170 MB/s (3위)
Naive	39 MB/s (4위)

요약

항목	최적화 전	최적화 후	개선
Bitslice (8블록)	3.3 MB/s	229.7 MB/s	69x
T-table 대비	0.02x	1.35x	-

핵심 포인트

1. AES-NI가 가장 빠름 - 하드웨어 명령어로 1.5 GB/s 이상
2. 최적화된 Bitslice가 T-table보다 35% 빠름 - 소프트웨어 구현 중 최고
3. Bitslice의 장점: Constant-time (사이드채널 공격 방어) + 높은 처리량
4. 1블록 단독 처리는 비효율 - transpose 오버헤드로 인해 Naive보다 느림

- Rijndael-256 구현 계획
 - ARM64
 - ARM Cryptographic Extension을 활용한 구현 완료
 - Fixslicing도 적용하여 성능 분석
 - Cortex-M4
 - Fixslicing 논문 분석 후 구현 예정
 - GPU
 - 현재 여러 메모리에 대한 성능 비교 완료, 다만 ECB 모드로 구현하여 예상한 결과랑 다르게 나옴.
 - CTR와 ES로 구현하여 다른 방향으로 성능 개선 분석 예정